

# Ohjelmistokehityksen sovellusprojekti (TVT Monimuoto)

---

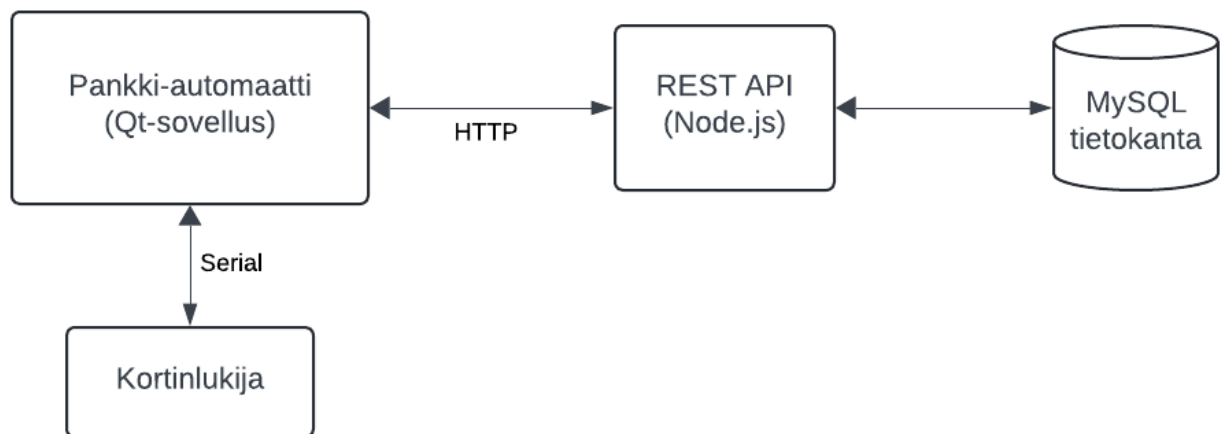
- Opiskelijoiden tehtävä on suunnitella ja toteuttaa pankkiautomaattijärjestelmä: **Yleisohje ja arviointi**
- 4 opiskelijan ryhmät määrittellään [Excel-dokumentissa](#)
- **Projektin alustaminen**
- Viikko-ohjelmat:  
[Viikko 1](#) | [Viikko 2](#) | [Viikko 3](#) | [Viikko 4](#) | [Viikko 5](#) | [Viikko 6](#) | [Viikko 7](#) | [Viikko 8](#) |

## Projektityön kuvaus

---

Työn aihe on pankkiautomaatti

Ohjelmiston rakenne on seuraava



Huom! Monimuotoryhmissä ei käytetä kortinlukijaa, vaan kortinnumero annetaan Qt-sovellukseen manuaalisesti.

Työ sisältää

- Tietokannan (MySQL/MariaDB)
- REST API:n (Node.js/Express.js)
  - Käytettävä MVC-mallia
  - Käytettävä callbackeja (ei Promisea, eikä async-await rakennetta)
  - Ei saa käyttää mitään ORM:ia
- Pankkiautomaattisovelluksen (Qt työpöytäsovellus, jossa käytetään Qt Network moduulia)
- Sovelluksen koodin GitHubissa
- Vaaditut dokumentit GitHubissa

**Huom!** Edellä mainitut kuuluvat kurssin sisältöön ja arviointi perustuu niiden osaamiseen, joten millään muilla tekniikoilla noita ei saa korvata.

## Sovelluksen toiminta

- Qt-sovellus kommunikoi REST API:n kanssa http-protokollan avulla.
- REST API hoitaa kommunikoinnin tietokannan kanssa.

## Oppimistavoitteet

---

- Opiskelija tunnistaa ja ymmärtää ohjelmistokehityksen vaihejakomallin perusvaiheet. Hän tietää eri vaiheiden merkitykset, vaihetuotteet ja vaiheiden erot
- Itsenäisen ja ryhmätyöskentelyn avulla opiskelija oppii suunnittelemaan ja toteuttamaan vaatimusmäärittelyn mukaisen järjestelmän käyttäen moderneja kehitystyökaluja
- Opiskelija ymmärtää ryhmätyöskentelyn merkityksen ohjelmistokehitystyössä
- Opiskelija osaa käyttää oliopohjaista mallinnuskieltä kehitystyön (UML) eri vaiheissa ja osaa kirjoittaa kaavioiden pohjalta ohjelmakoodia
- Opiskelija osaa suunnitella ja toteuttaa oliopohjaisen sovelluksen luokkakirjaston mukaisesti
- Opiskelija osaa suunnitella ja toteuttaa sovellukseen tietokanta-arkkitehtuurin
- Opiskelija osaa laatia ohjelmistoprojektin dokumentaation ja pystyy viestimään suullisesti ja kirjallisesti, myös englanniksi

## Opiskelijan arviointi

---

Kukin opiskelija arvioidaan yksilöllisesti ja arvioinnissa huomioidaan seuraavat asiat:

- Sovelluksen arvosana
- Vertais- ja itsearviointi
- Ohjaajien näkemys
- Githubin informaatio

## Vertaisarvioinnin kohteet

- Ryhmätyöskentely
- Itsenäinen työ
- Projektisitoutuminen
- Qt-ohjelmointi
- REST API -ohjelmointi
- Tehtävien vaikeustaso
- Gitin käyttö

## Sovelluksen arviointi

Arviointi perustuu tähän dokumenttiin. Mikäli ristiriitaista tietoa esiintyy, niin tämä dokumentti on se, jota noudatetaan.

## Vähimmäisvaatimukset sovellukselle (arvosana 1)

- Debit kortti toteutettava:
  - Debit-kortti kytketään tiliin, jolla ei ole luottoa eli tilin saldo ei saa mennä miinukselle
- Qt-sovelluksen aloituskäyttöliittymä
- Oikealla PIN-koodilla avautuu pääkäyttöliittymä, väärällä uudelleenkysely
- Saldon tarkastelu
- Rahan nosto: 20, 40, 50 tai 100 €
- Näytetään 10 viimeisintä tilitapahtumaa

## Vähimmäisvaatimukset (arvosana 2)

- Webtoken autentikointi toteutettu
- PIN-koodin syötön aikaraja 10 sekuntia (jos koodia ei anneta 10 sekunnin aikana palataan aloituskäyttöliittymään)
- REST API:in on toteutettu kaikkien tietokanta-taulujen CRUD-operaatiot (vaikkei niitä tarvita pankkiautomaatissa)

## Hyvän arvosanan vaatimukset (arvosana 3)

- Debit-kortin lisäksi on toteutettava credit-kortti:
  - Credit-kortti liitetään credit tiliin eli tilillä on luottoraja ja saldo saa mennä miinukselle luottorajan verran
- Vapaavalintaisen summan nosto (automaatissa vain 20 ja 50 € seteleitä)
- Kolme väärää PIN-koodia lukitsee kortin (ei vaadita tallentamista tietokantaan)

## Hyvän arvosanan vaatimukset (arvosana 4)

- Korttilukitus tallennetaan tietokantaan (eli lukitus säilyy vaikka sovellus käynnistetään uudelleen)
- 30 sekunnin inaktiivisuus palauttaa alkutilaan (jos käyttäjä ei tee mitään 30 sekunnin aikana, palataan aloituskäyttöliittymään ja kaikki muut ikkunat suljetaan)
- Tilitapahtumien selaus (eteen/taakse, 10 tapahtumaa kerrallaan)

## Kiitettävän arvosanan vaatimukset (arvosana 5)

- Toteutetaan kaksioiskortti:
  - Kortilla on debit ja credit ominaisuus eli se on kytketty yhteen credit-tiliin ja yhteen debit-tiliin
- Kirjautuessa valinta: debit vai credit:
  - Valinta tehdään vain jos kyseessä kaksoiskortti
  - Jos kyseessä debit-kortti tai credit-kortti, valinta ohitetaan
- Tilakaavio luotu
- **Lisäominaisuus** sovittava ohjaajan kanssa

## Tiivistelmä arvosanoille

Nämä ovat ohjelmistokokonaisuutta projektihallinnallisesta näkökulmasta koskevat minimi (arviointi):

|                                 | 1 | 2 | 3 | 4 | 5 |
|---------------------------------|---|---|---|---|---|
| Versionhallinnan käyttö         | x | x | x | x | x |
| Viikkopalaverit                 | x | x | x | x | x |
| Toiminnallinen määrittelydokum. | x | x | x | x | x |
| Tekninen määrittelydokum.       | x | x | x | x | x |
| ER-kaavio                       | x | x | x | x | x |
| Readme.md                       | x | x | x | x | x |

Nämä ovat itse ohjelmistokokonaisuutta koskevat minimiit (arviointi):

|                            | 1 | 2 | 3 | 4 | 5 |
|----------------------------|---|---|---|---|---|
| Kirjautuminen PIN-koodilla | x | x | x | x | x |
| Saldon näyttö              | x | x | x | x | x |
| Rahan nosto (20,40,50,100) | x | x | x | x | x |
| Tilitapahtumien näyttö     | x | x | x | x | x |
| Debit kortti               | x | x | x | x | x |
| Webtoken autentikointi     | x | x | x | x | x |
| PIN-koodille 10 s timer    | x | x | x | x | x |
| Kaikki CRUD-operaatiot     | x | x | x | x | x |
| Credit kortti              |   | x | x | x |   |
| Rahan nosto (muu summa)    |   | x | x | x |   |
| PIN-lukitus istunnolle     |   | x | x | x |   |
| PIN-lukitus tietokantaan   |   |   | x | x |   |
| 30 s timerit               |   |   | x | x |   |
| Tilitapahtumien selaus     |   |   | x | x |   |
| Tilakaavio                 |   |   |   | x |   |
| Kaksoiskortti              |   |   |   | x |   |
| Lisäominaisuus             |   |   |   | x |   |

### Arvosanaa alentavia seikkoja

- Dokumentoinnin puutteet
- Sovelluksen rakenne ei ole annettujen määritysten mukainen
- Poissaolo palaverista

## Tiivistelmä arvioinnissa huomioitavista asioista:

- Aikataulussa pysyminen. Työtä pitää tehdä järjestelmällisesti. Viikkoraportointi vaaditaan!
- Jokaisen ryhmän jäsenen pitää osata kertoa omasta tekemisestä viikkopalaverissa
- Opiskelijan tulee osata selittää kirjoittamansa koodi
- Ohjaajan arvio perustuu palaverissa saatuihin kokemuksiin ja GitHubin näkymiin
- Ryhmän tuottaman sovellukseen tasoon (kts. Sovelluksen arviointi)
- Toveriarvio tehdään web-sovelluksella (vertaisarviointi)
- Itsearvio tehdään web-sovelluksella (itsearviointi)
- Vaaditut dokumentit (heikko dokumentointi voi alentaa arvosanaa)
- Englanninkielinen poster (hyväksytty/hylätty, pitää päästä läpi)
- Loppuesitys vaikuttaa arvosanaan
- Arvosanaa ei voi korottaa myöhemmin

## Oppimateriaalit

---

### Qt/Express-materiaalit (Pekka Alaluukas)

- Pekka Alaluukkaan [ohjeet ja tallenteet videosoittolistana](#)
- Git perusteita [Peatutor.com/git\\_tutor/](https://www.peatutor.com/git_tutor/)
- Ohjelmistotuotanto [Peatutor.com/swengineering/](https://www.peatutor.com/swengineering/)
- Muita Pekan tekemiä ohjeita (Qt yms.): [Peatutor.com/](https://www.peatutor.com/)

### Teemaluentoja: AI, IaC, CI/CD, reverse proxy, ohjelmistolisensseistä (Teemu Korpela)

- 7.1.2026 - Tiedonhausta, IT-uutisista, alan seuraamisesta ja Kanban-taulut
- 15.1.2026 - Verkkopalveluista, IaC, CI/CD, rajapinnoista, reverse proxyt
- 21.4.2021 - Ohjelmistolisensseistä (aikaisempi tallenne, ei tarvetta uudelle)
- 29.1.2026 - AI-(vibe)koodaus

### Ohjelmistokehityksen perusteet ja UML-mallinnus videot Yujassa (Teemu Leppänen)

- Teemu Leppäsen luentotallenteet [videosoittolista \(kevät 2025\)](#)

### Kaaviot dokumentointiin

Esimerkiksi näillä työkaluilla:

- Drawio: <https://www.drawio.com/>. Suora linkki: <https://app.diagrams.net/>
- Lucidchart: <https://www.lucidchart.com>
- Diagrameditor: <https://www.diagrameditor.com/>
- PlantUML: <https://plantuml.com/>
- Mermaid: <https://mermaid.js.org/>

Suosittellemme Mermaidia.

Esimerkkejä Mermaidilla tehtyihin kaavioihin: [https://peatutor.com/swengineering/mermaid\\_example.php](https://peatutor.com/swengineering/mermaid_example.php)

# Vaatimukset dokumenteille

---

- Vaadittavat dokumentit: toiminnallinen määrittely ja tekninen määrittely
- Luettavissa GitRepon documents-kansiosta
- Kirjoitettu Markdownilla
- Kaaviot lisättynä dokumentteihin (suositellaan Mermaidia)
- Lisäksi README.MD tiedostossa vapaamuotoinen kuvaus sovelluksesta

## Vaatimukset tietokannalle

---

### Ilman credit-kortti ominaisuutta

- Useita tilejä asiakkaalla
- Yhdellä tilillä yksi omistaja
- Asiakkaalla voi olla tili ilman korttia
- Useita kortteja asiakkaalla, mutta yksi kortti → yksi tili
- Asiakastiedoissa: etunimi, sukunimi, osoite
- PIN-koodi hashattuna (bcrypt)

### Kun toteutetaan credit-kortti ominaisuus

- Credit-korteilla pitää olla luottoraja (credit-korteille ei tarvita erillistä taulua, jos debit-korteille laitetaan luottorajaksi nolla)

### Kun toteutetaan kaksoiskortti

- Kortilla pääsy useaan tiliin (debit ja credit)

### Lisäominaisuuksia tietokannalle

- Asiakkaalla käyttöoikeus toisen omistajan tilille

### Tileistä ja korteista

- Vaikka tässä tehdään pankkiautomaatti, niin tehdään tietokannasta kuitenkin oikeaa pankintietokantaa muistuttava. Eihän pankeilla ole erikseen tietokantaa pankkiautomaattien tileille. Siksi siis pitää voida luoda tilejä ja osalle niistä annetaan kortti osalle ei.
- Sellainen kortti, jossa on sekä debit, että credit ominaisuus toimii niin, että se on kytketty kahteen tiliin:
  - toinen on debit tili (se on asiakkaan oma tili)
  - toinen tili on credit tili (sen omistaa pankki ja asiakas ei näe sitä tiliä verkkopankissa)
  - tässä on siis kyseessä **monen-suhde-moneen yhteys**:
    - yhdelle tilille voi olla pääsy monella kortilla: vaikkapa koko perheellä
    - yksi kortti on kytketty moneen eri tiliin (vaikka se on käytännössä korkeintaan kahteen tiliin(debit ja credit).

# Viikkopalavereiden yleinen agenda

---

- Pääsääntöisesti kaikkien pitää olla paikalla
- Yleistä keskustelua, että miten projekti on edennyt
- Kukin opiskelija kertoo (ja näyttää) mitä on tehnyt kuluneen viikon aikana
- Versiohallinnan esittely
- Muutoksia arvosanatavoitteeseen tai tavoitteisiin ylipäättänsä

## Viikko 1

---

### 1. Päivän / TEHTÄVÄT

1. Luodaan neljän hengen ryhmät [Excel-dokumentissa](#)
2. Jokainen opiskelija luo tunnuksen itselleen sivustolla [https://peatutor.com/project\\_app/register/tvt26kmo](https://peatutor.com/project_app/register/tvt26kmo)
  - Voit keksiä minkä hyvänsä tunnuksen (joka on vapaa)
  - Rekisteröityä voi vain oamk.fi ja oulu.fi sähköposteilla
  - Luotuaasi tunnuksen, saat sähköpostin, jossa on tunnuksesi ja salasanasasi. Pidä ne tallessa.

**Huom!** Tarkista ennen rekisteröitymistä, tarkista mikä on sinun GitHub-tunnus, koska se on annettava rekisteröityessä.

3. Jokaisesta ryhmästä yksi luo kurssin Teams-kanavan **ALAISUUTEEN** (ei siis kokonaan uutta Teams-kanavaa) uuden PRIVAATIN alikanavan, jolla on sama nimi kuin ryhmällä Excelissä eli group\_1, group\_2, ....

### Loppuviikon / TEHTÄVÄT

- Tutustukaa arviointikriteereihin ja päättäkää mihin arvosanaan pyritään
- Tarkista että olet kurssin Teams-kanavalla (pyydä opettajalta pääsy jos et ole). Käytä students.oamk.fi-sähköpostiosoitetta kun kirjaudut Teamssiin
- Github käyttöön (Pekan tekemän organisaation alle): [Pekan ohje](#)
- Ryhmän jäsenet sopii alustavasti kuka tekee mitäkin toiminnallisuuksia (mutta ei niin, että vain yksi tekee koko Qt-työpöytäsovelluksen, että vain yksi tekee koko tietokannan jne.)
- Ryhmä sopii käytetäänko Qt sovelluksessa build järjestelmänä **qmake**:a vai **cmake**:a (on parasta että koko ryhmä käyttää samaa)
- Aloittakaa tekemään vaatimusmäärittelydokumenttia (pitää tehdä yhdessä). Dokumentti tehdään GitRepon documents kansioon nimellä toiminnallinen.MD.

- Aloittakaa tekemään teknistä määrittelydokumenttia (pitää tehdä yhdessä). Dokumentti tehdään GitRepon documents kansioon nimellä tekninen.MD.
- Katsokaa yhdessä valmiiksi viikon 2+ tavoitteet
- Tämän viikon aikana pitää olla tehtynä:
  - Github repository käyttöön
  - Toiminnallinen määrittely-dokumentti valmis
  - Tekninen määrittely-dokumentti alulle

### Vinkkejä tietokannan suunnitteluun

- Lukekaa <https://peatutor.com/databases/db.php#design> ja miettikää erityisesti **monen-suhde-moneen yhteydet**
- Miettikää tietotyyppejä ja tässä apuna <https://peatutor.com/databases/mysql.php#types>

## Viikko 2

---

- Viikkopalaveri opettajan kanssa
  - Versiohallinnan esittely (Tarkistetaan että repository on alustettu ja vaatimusmäärittely valmis)
- Tekninen määrittely valmiiksi
- Luodaan MySQL-tietokanta
  - generoidaan model MySQL Workenchillä
  - tarkistetaan että Workbenchillä saatu ER-model vastaa toiminnallisen määrittelyn diagrammia
  - ladataan Workbenchin ER-modelista Exportattu PNG-kuva GitHubiin documents-kansioon
  - Ilmoitus Pekalle, että tarkista
  - Workbenchillä synkronoidaan modelista tietokanta
- Sovelluksen tekemistä : aloittaakaa backendistä
  - kukin opiskelija valitsee tietokannasta yhden taulun, jolle alkaa tekemään CRUD-operaatioita
- Tämän viikon aikana pitää olla tehtynä:
  - MySQL tietokanta tehtynä ja Workbenchin ER-modelin kuva GitHubissa
  - Tekninen määrittely pääosin valmis
  - Backendissä koodia

## Viikko 3

---

- Viikkopalaveri
  - CRUD-operaatioista demo (Pitää olla jotain endpointteja backendissä)
- Kirjoita Github-projektille kuvaus markdownilla (readme.md-tiedosto). Github osaa prosessoida markdown-kieltä suoraan readme.md:stä HTML:ksi
  - Muista päivittää omaa projektikuvausta Githubissa (readme.md) myös myöhemmin!
  - Esimerkkejä [hyvistä readme-projektitiedostoista](#)
  - Teemun tekemä yksinkertainen esimerkki: <https://github.com/t2946282/demoproject>

- Sovelluksen tekemistä
- Tämän viikon aikana pitää olla tehtynä:
  - Readme.md:n ensimmäinen versio repositorylle Githubissa
  - Backendissä endpointeilla jotakin toimintaa
  - Tehtyjen endpointtien testausta [Postmanilla](#)

## Viikko 4

---

- Viikkopalaveri
  - Versiohallinnan esittely
- Sovelluksen tekemistä
- Tämän viikon aikana pitää olla tehtynä:
  - Login endpoint backendissä (kortin numerolla ja oikealla PIN koodilla saadaan webtoken)
  - Vähintään pankkiautomaatin tarvitsemat endpointit backendissä

## Viikko 5

---

- Viikkopalaveri
  - Versiohallinnan esittely
  - Nyt pitää olla jo Qt-sovelluksessa jotain omaa koodia
- Sovelluksen tekemistä
- Tämän viikon aikana pitää olla tehtynä:
  - Kirjautuminen onnistuu Qt-sovelluksesta

## Viikko 6

---

- Viikkopalaveri
  - Versiohallinnan esittely
  - Sovelluksen esittely
- Tämän viikon aikana pitää olla tehtynä:
  - Qt sovelluksessa kirjautumisen jälkeen jotain toimivaa

## Viikko 7

---

- Viikkopalaveri
  - Versiohallinnan esittely
- Sovelluksen tekemistä
- Demovideon valmistelu
- Ryhmä tekee yhdessä posterin englanniksi. Posteripohja löytyy Teamssista

- Ota posterista hyvälaatuinen kuvaruutukaappaus, lisää se kuvana Github-repositoryyn ja linkitä näkyväksi readme.md tiedostossa repositoryn etusivulla
- Tämän viikon aikana pitää olla tehtynä:
  - Posterin valmiiksi ja Teamssiin
  - Posterin Githubissa kuvana ja linkitetty readme.md:ssä repositoryn etusivulle

## Viikko 8

---

- Laadi vastaava taulukko kuin kohdassa **Tiivistelmä arvosanoille** ja rastita siihen oman toteutuksen suoritettut tehtävät.
  - Voit ladata excel-tiedoston (taskit.xlsx) Teamsin kanavalta **Tiedostot ja yhteiset oppimateriaalit**
  - Rastita tehdyt tehtävät
  - Lataa tiedosto GitRepon juureen (jos et käytä exceliä laita kuitenkin nimen alkuosaksi taskit)
- Demovideo projektista:
  - Videon pituuden tulisi olla noin 5 minuuttia, missä ehtii yleensä näyttämään keskeiset osat applikaatiosta ja posterista.
  - Videon on oltava julkisesti saatavilla ilman kirjautumista
    - YouTubeen unlisted-videoksi (myös students.oamk.fi -tunnukset toimivat myös Youtubeen)
    - Älä aseta videon lupaa "YouTube-sisältö lapsille", koska se ei salli videon tallentamista YouTube-soittolistaan
    - Linkkaa videon URL ryhmän Teams-kanavalle
  - Luo 3-4 sivun PowerPoint- tai PDF-dokumentti tukemaan videon esitystä. Dokumentissa tulisi olla vähintään:
    - Mitkä olivat projektin tavoitteet
    - Tiivistelmä arvosanoille -taulukko
    - Ketkä osallistuivat projektiin ja mitä he tekivät (suunnilleen)
    - Mikä oli hyvää, mikä oli huonoa
    - Esitä dokumentin sisältö videon alussa
    - Kaikkien ei välttämättä tarvitse puhua videolla (mutta toki saa)
    - Näytä posterin videon lopuksi
    - Lisää PowerPoint- tai PDF-dokumentti Github-repositoryyn
  - Esittele pankkiautomaattiprojekti
- Loppuesitykset koko luokalle (osallistumispakko)
  - Ohjelman demonstrointi ja vapaata keskustelua
  - Posterin esittely

## Projektin alustaminen

---



Voit katsoa ohjevideon osoitteesta:

[https://www.youtube.com/watch?v=\\_lfn6vsrOJY](https://www.youtube.com/watch?v=_lfn6vsrOJY)

---

## 1. Repositoryn alustaminen

Yksi ryhmän opiskelijoista alustaa GitHub-repositoryn seuraavasti:

```
# Kloonaa repon omalle koneelleen
git clone <repository-url>

cd groupx # jossa groupx on kloonattu kansio ja x oman ryhmän numero
git checkout -b initialize

mkdir documents
```

---

## 2. Backendin alustaminen

Anna groupx kansiossa seuraavat komennot

```
mkdir backend
cd backend
npx express-generator --no-view
npm install
```

**Huom!** Tuo express-generator asentaa hieman vanhat npm-paketit, joten voitte halutessanne korvata tuon npx komennon seuraavilla komennoilla (jotka ajetaan backend kansiossa):

```
npm init
npm install express mysql2 bcryptjs jsonwebtoken dotenv
mkdir routes
mkdir models
```

Ja sitten app.js rakennetaan kuten luennoilla on opastettu.

---

## 3. Qt-sovelluksen alustaminen

1. Käynnistä **Qt Creator**
2. Luo **Qt Widget** -tyyppinen sovellus, jonka nimeksi **bank-automat**
3. Tallenna sovellus kansioon **groupx**
4. Käännä sovellus
5. Tarkista, että **bank-automat**-kansion alle ilmestyi **build**-kansio
6. Jos **build**-kansiota ei ilmestynyt:

- Poista **bank-automat**-kansio
- Tarkista Qt:n asetukset:  
[https://peatutor.com/c\\_kieli/qt\\_asennus.php](https://peatutor.com/c_kieli/qt_asennus.php)
- Luo sovellus uudestaan

---

## 4. **.gitignore**-tiedoston luominen

Luo tiedosto projektikansion **groupx** juureen ja kirjoita siihen seuraavat rivit:

```
backend/node_modules/  
bank-automat/build/  
bank-automat/.qtcreeator/  
bank-automat/*.user
```

---

## 5. Muutosten lisääminen ja pushaaminen

Suorita komennot kansion **groupx** juuressa:

```
git add .  
git commit -m "projekti alustettu"  
git push origin initialize
```

---

## 6. Tarkistukset GitHubissa

Varmista, että GitHubissa näkyy seuraavat kansiot:

- backend
- bank-automat
- documents

Ja että seuraavat **eivät ole GitHubissa**:

- backend/node\_modules
- bank-automat/build
- bank-automat/.qtcreeator
- bank-automat/xxx.user

---

## 7. Pull Request

- Jos kaikki edellä meni oikein, tee **Pull Request**
- Pyydä jotain muuta ryhmän jäsentä hyväksymään PR ja yhdistämään **initialize** branchin **mainiin**

## 8. Branchin yhdistämisen jälkeen

Henkilö, joka teki alustusvaiheet

- suorittaa komennot:

```
git checkout main  
git pull origin main
```

- ja tämän jälkeen hän luo oman branchin

Muut ryhmän jäsenet

- kloonavat repositoryn
- luovat oman branchin

## Lisäominaisuusideoita

---

(arvosanan 5 tarvitaan vähintään yksi tällainen lisäominaisuus)

### Kuvan lataus ja näyttäminen

- Kuvan lataaminen backendiin ja näyttäminen Qt-sovelluksessa (vaikutus arvosanaan 1)

Idean esittelyvideo: <https://www.youtube.com/watch?v=DIKRIZTNYI8>

Toimintaperiaate:

- Tietokanta taulussa on tekstikenttä, johon tulee kuvan nimi (esim. `aku.jpg`).
- Kuva ladataan REST API:n kansioon (yleensä `public`-kansioon).
- Kuva kansioon pitää päästä esim. selaimella.
- Qt-sovelluksessa kuva näytetään `Label`-komponentissa.

REST API:ssa voi käyttää [Multer-moduulia](#).

### Swagger dokumentointi

- Lisätään sovellukseen swagger-sivu (vaikutus arvosanaan 1)

Idean esittelyvideo: <https://www.youtube.com/watch?v=M6Fj5Y2K24w>  
<https://www.npmjs.com/package/swagger-ui-express>

### Logitus

- Tapahtumien logittaminen backendissä ja niiden näyttäminen jollakin tavalla (`morgan`-moduuli). Pelkkä logitus on aika helppo, joten sen vaikutus n. 0,5. Mutta jos keksitte siihen jotain lisää, niin sitten isompi vaikutus.

# WebSocket

Toteutetaan WebSoketeilla jokin toiminto sovellukseen (vaikutus arvosanaan 1).

- Node.js WebSocket: <https://www.npmjs.com/package/ws>
- Qt:n websocket-moduuli

Idean esittely: <https://youtu.be/QGnv7s0Jllo>

# Docker

Sovelluksen ajaminen Dockerissa (vaikutus arvosanaan 1).

- <https://youtu.be/DseMnAW0OTk>

# Testien lisääminen backendiin

Esimerkiksi **jest** ja **supertest** (vaikutus arvosanaan 1)

Esittelyvideo: <https://youtu.be/HEZufcp2uml>

Tai Newman

Esittelyvideo: <https://youtu.be/Wvv8GWQdvKU>

# CI/CD

- Jonkinlainen yksinkertainen CI/CD tai ainakin CD ( esim. backendin julkaisu jossain pilvipalvelussa ja qt-sovelluksen "releasen" automatisointi Githubiin tai toiselle palvelimelle ladattavaksi vaikka Github actioneilla) (vaikutus arvosanaan 1)

# Kafka

-Apache Kafkan käyttö järjestelmien väliseen viestinvälitykseen. Kafka voidaan ottaa käyttöön esimerkiksi Aivenin pilvipalvelussa.

-Ota Kafka käyttöön ja rakenna Node.js-sovellus, joka vastaanottaa Kafka-viestejä ja näyttää ne käyttäjälle.

-Pekan esimerkkivideo: <https://youtu.be/Cf4B9XYzq7k>

# Verkkopankin toteuttaminen

- Verkkopankin toteuttaminen (vaikutus arvosanaan 1)

# Ylimääräinen Qt-sovellus

- Qt-sovellus pankin henkilökunnalle. Sovelluksella voidaan esimerkiksi luoda uusia asiakkaita, tilejä ja kortteja jne.

# Generatiiviset tekoälyt (AI-koodaus) ja vastaavat apuvälineet. Ohjaajien (ja yleisestikin IT-opettajien) ajatuksia aiheesta:

---

- Tekoäly on hyvä renki, mutta huono isäntä. Varsinkin oppimisessa.
- Tekoälyäkin pitää oppia hyödyntämään, mutta vähän myöhemmin
- Ensin on kuitenkin syytä opiskella perusteet, oli se sitten vaikkapa IT arkkitehtuurista, ohjelmistotekniikan perusteista, tietoverkoista, tietoturvallisuudesta, tietosuojasta, dokumentoinnoista, elektroniikasta yms.
- Työnantajat tuskin palkkaavat tuhansia euroja kuussa maksavaa työntekijöitä, jotka ovat pelkästään tekoälykonttoristeja
- Perusasioiden ymmärrys ei katoa mihinkään ja onhan se myös ammattiylpeyttä suunnitella ja käsittää mitä tapahtuu milloinkin
- Me ohjaajina emme halua arvioida tekoälyn tekemää sovellusta ja tekemistä, vaan opiskelijoiden. Emme myöskään ryhdy poliisiksi, joka käyttää työaikansa tekoälyn jäljittämiseen, vaan **opiskelijalla on oltava itsellään halu oppia eikä tavoitella pelkästään arvosanoja**
- Tämän projektikurssin ohjaajia yhdistää vuosikymmeniä kestänyt innostus ja kiinnostus tietotekniikkaan ja uteliaisuus oppia ja kokeilla uutta. Myös tekoälyalustoja, jotka on vain uusi mielenkiintoinen vaihe tietotekniikan historiassa. Emme todellakaan ole tekoälyvastaaisia, vaan päin vastoin. Niitä on hyvä ja tärkeää oppia hyödyntämään, mutta ei siten että perusteet jää oppimatta!

## Softalisensseistä

---

- Choose a license: <https://choosealicense.com/>
- Public license selector: <https://ufal.github.io/public-license-selector/>

## Kirjat ja kurssit taustatiedoksi ja malliksi

---

Tee tunnus O'Reillyn verkkokirjastoon [students.oamk.fi:n](https://libguides oulu.fi/oreilly) sähköpostilla: <https://libguides oulu.fi/oreilly> ja valitse institution not listed. Tuo on kaupallinen palvelu, mihin Oamkin kirjasto on ostanut pääsyn. Kannattaa käydä selailemassa tuota online-kirjastoa muutenkin.

Aika tunnettuja ja arvostettuja ohjelmistotekniikan kirjoja. Enemmistö näistä kirjoista suoraan tästä [tweetistä](#):

- The Pragmatic Programmer: your journey to mastery: <https://learning.oreilly.com/library/view/the-pragmatic-programmer/9780135956977/>
- Code Complete: <https://learning.oreilly.com/library/view/code-complete-2nd/0735619670/>
- Design Patterns: Elements of Reusable Object-Oriented Software: <https://learning.oreilly.com/library/view/design-patterns-elements/0201633612/>
- Designing Data-Intensive Applications: <https://learning.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/>

- Clean Code Fundamentals: <https://learning.oreilly.com/course/clean-code-fundamentals/9780134661742/>
- Clean Architecture: A Craftsman's Guide to Software Structure and Design: <https://learning.oreilly.com/library/view/clean-architecture-a/9780134494272/>
- Modern Software Engineering: Doing What Works to Build Better Software Faster: <https://learning.oreilly.com/library/view/modern-software-engineering/9780137314942/>
- Algorithms: 24-part Lecture Series: <https://learning.oreilly.com/course/algorithms-24-part-lecture/9780134384528/>
- Fundamentals of Software Architecture: <https://learning.oreilly.com/library/view/fundamentals-of-software/9781492043447/>

Qt-aiheiset (ei tiedoa laadusta):

- Qt 6 C++ GUI Programming Cookbook: <https://learning.oreilly.com/library/view/qt-6-c/9781805122630/>
- Getting Started with Qt 5: <https://learning.oreilly.com/library/view/getting-started-with/9781789956030/>

Node.js (ei tieto laadusta):

- Modern JavaScript from the Beginning: <https://learning.oreilly.com/course/modern-javascript-from/9781805127826/>
- Node.js - The Complete Guide: <https://learning.oreilly.com/course/node-js-the/9781838826864/>
- Node.js Design Patterns: <https://learning.oreilly.com/library/view/node-js-design-patterns/9781839214110/>
- The Complete Node.js Developer Course: <https://learning.oreilly.com/course/the-complete-node-js/9781789955071/>

Linux-aiheiset (ei tieto laadusta):

- How Linux Works, 3rd Edition: <https://learning.oreilly.com/library/view/how-linux-works/9781098128913/>
- Learning Modern Linux: <https://learning.oreilly.com/library/view/learning-modern-linux/9781098108939/>
- Efficient Linux at the Command Line: <https://learning.oreilly.com/library/view/efficient-linux-at/9781098113391/>

UML (ei tietoa laadusta):

- Learning UML 2.0: <https://learning.oreilly.com/library/view/learning-uml-2-0/0596009828/>
- UML Fundamentals: <https://learning.oreilly.com/course/uml-fundamentals/9781771373630/>
- Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development: <https://learning.oreilly.com/library/view/applying-uml-and/0131489062/>

API (ei tietoa laadusta):

- Mastering API Architecture: <https://learning.oreilly.com/library/view/mastering-api-architecture/9781492090625/>

- Understanding APIs and RESTful APIs: <https://learning.oreilly.com/course/understanding-apis-and/9781800564121/>

MySQL (ei tietoa laadusta):

- SQL Queries for Mere Mortals: A Hands-On Guide to Data Manipulation in SQL: <https://learning.oreilly.com/library/view/sql-queries-for/9780134858432/>
- Learning SQL, 3rd Edition: <https://learning.oreilly.com/library/view/learning-sql-3rd/9781492057604/>
- Fundamentals of Data Engineering: <https://learning.oreilly.com/library/view/fundamentals-of-data/9781098108298/>
- MySQL 5: <https://learning.oreilly.com/course/mysql-5/9781926873961/>

## Ohjaajien kommentteja ja vinkkejä

---

- Tähän on kasattu aikaisempien projektien [ohjaajien kommentteja ja vinkkejä](#)